

Efficient 3D Hilbert Curve Encoding and Decoding Algorithms

JIA Lianyin^{1,2}, LIANG Binbin¹, LI Mengjuan³, LIU Yong⁴, CHEN Yinong⁵, and DING Jiaman^{1,2}

(1. Faculty of Information Engineering and Automation, Kunming University of Science and Technology, Kunming 650500, China)

(2. Yunnan Key Laboratory of Artificial Intelligence, Kunming University of Science and Technology, Kunming 650500, China)

(3. Library, Yunnan Normal University, Kunming 650500, China)

(4. School of Information Science and Engineering, Guangxi University for Nationalities, Nanning 530006, China)

(5. School of Computing, Informatics, and Decision Systems, Arizona State University, Tempe 85287, USA)

Abstract — Hilbert curve describes a one-to-one mapping between multidimensional space and 1D space. Most traditional 3D Hilbert encoding and decoding algorithms work on order-wise manner and are not aware of the difference between different input data and spend equivalent computing costs on them, thus resulting in a low efficiency. To solve this problem, in this paper we design efficient 3D state views for fast encoding and decoding. Based on the state views designed, a new encoding algorithm (JFK-3HE) and a new decoding algorithm (JFK-3HD) are proposed. JFK-3HE and JFK-3HD can avoid executing iteratively encoding or decoding each order by skipping the first 0s in input data, thus decreasing the complexity and improving the efficiency. Experimental results show that JFK-3HE and JFK-3HD outperform the state-of-the-arts algorithms for both uniform and skew-distributed data.

Key words — 3D Hilbert curve, Space-filling curve, state view, JFK-3HE, JFK-3HD.

I. Introduction

Space-filling curve (SFC), also called Peano curve, was first proposed by Peano in 1890^[1]. Hilbert^[2] popularized it and proposed the first general generation method. Z curve and Hilbert curve are the most commonly used SFCs. Compared with the other curves, Hilbert curve has much better spatial cohesion^[3], spatial continuity, and self-similarity^[4] and has been widely used in spatial query processing^[5–8], image processing^[9–11], load balancing^[12–14], Adaptive Steganography^[15] and

many other fields.

Despite the advantages mentioned above, the encoding and decoding processes of Hilbert curve are much more complicated than the Z curve, which limits its application to a certain extent. Encoding and decoding are the bases for many Hilbert space-filling curve based applications. Increasing the encoding and decoding speed can improve the efficiency of these applications. For a long time, researchers paid more attention on 2D Hilbert curve, and many efficient encoding and decoding algorithms were proposed, e.g., Butz^[16], Cao^[17], Li^[18], BURKARDT^[19]. But these algorithms are usually designed for 2D coordinates, 2D image processing or some other related scenarios, which are difficult or inefficient for them to be extended to 3D. As 3D Hilbert curve are commonly used in fields like 3D images processing^[20], 3D Data sorting^[21], etc. It is of great importance to improve the encoding and decoding efficiency of 3D Hilbert curve.

Most earlier 3D Hilbert curve encoding and decoding algorithms, such as Sagan^[22], Millar^[23], are recursive based algorithms, which spend a lot of time on calculating one-to-one mapping, thus leading to a low efficiency^[24]. Considering of this, most recent algorithms turn to iteration rather than recursion. Moore *et al.*^[25] proposed bit-operation based non-recursive encoding and decoding algorithms, which can efficiently support arbitrary dimensional encoding and decoding. Li *et al.*^[26] designed a gene based static evolution rule table, which

transforms the input coordinates according to the rule table, thereby achieving efficient encoding. Zhang *et al.*^[27] proposed an efficient generalized 3-D Hilbert curve scanning algorithm, which uses 2 simple lookup tables instead of complex recursion, thus reducing encoding complexity. However, it mainly aims at the cases where the lengths of 3 dimensions in 3D space are not equal or not the power of 2.

The algorithms mentioned above have 2 weaknesses: 1) Most of the traditional algorithms are recursion based, which is not efficient. 2) The existing algorithms are not aware of the difference between different input data and thus spend equivalent computing costs on them. To solve these problems, we designed efficient state views, which can transform the recursive operation into iteratively querying the state views. Moreover, we found and proved that it is not necessary to iteratively encode and decode each order. Instead, we skip encoding and decoding the first k orders of input data where their bits are all 0s, which can reduce the number of orders needed to be actually encoded and decoded, thus reduce the complexity of the algorithms. Based on these ideas, we proposed JFK-3HE and JFK-3HD, an efficient 3D Hilbert curve encoding and an efficient 3D decoding algorithm, which can significantly improve the efficiency of the existing encoding and decoding algorithms.

II. 3D Hilbert Curve

3D Hilbert space filling curve (3DHC) has the characteristics of FASS (space filling, self-avoiding, simple and self-similar) and can be viewed as the locus of points that passes through all points once and only once in a 3D space. 3DHC equally divides a space in a layer manner from 3 dimensions. A 3DHC of order 1 equally divides the space into 8 subspaces, each of which has its unique coordinate and Hilbert code. A diagram of 3DHC of order 1 can be seen in Fig.1.

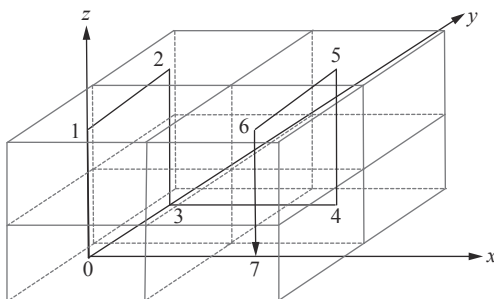


Fig. 1. A diagram of 3DHC of order 1

For example, the corresponding Hilbert code of the coordinate (1, 0, 1) in Fig.1 is 6. A 3DHC of order 2 can be obtained by embedding a 3DHC of order 1 into each subspace as shown in Fig.2. We can get a 3DHC of

order k by further dividing each subspace in a similar way. A 3DHC of order k divides the whole space into $2^k \times 2^k \times 2^k$ subspaces.

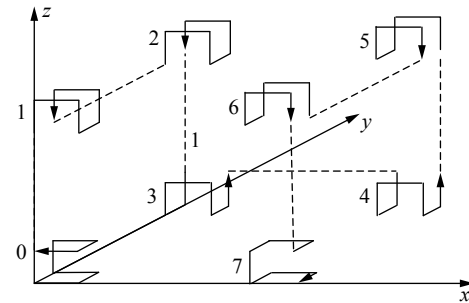


Fig. 2. A diagram of 3DHC of order 2

A 3DHC of order 1 has a total of 12 basic states named from 0 to 11 as shown in Fig.3.

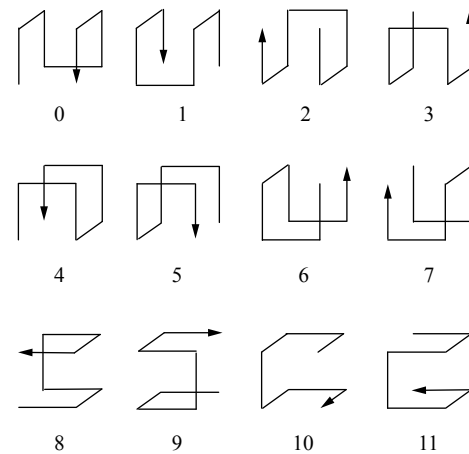


Fig. 3. 12 basic states of 3DHC of order 1

To better describe this paper, we first give some terms that will be used later. A basic description of Hilbert encoding and decoding is shown in Fig.4.

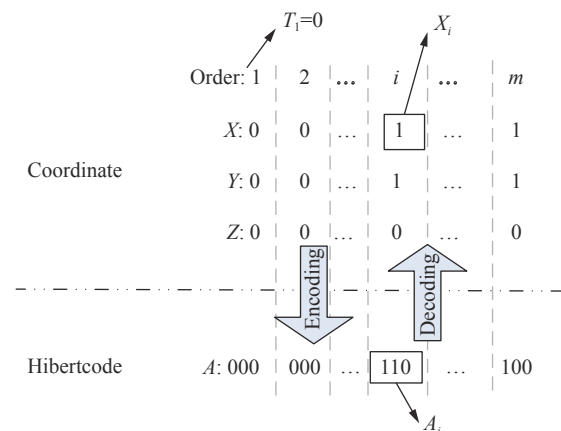


Fig. 4. A basic description of Hilbert encoding and decoding

In this paper, given a 3D coordinate $P(X, Y, Z)$, the binary representation of X is $X_1X_2 \dots X_i \dots X_{m-1}X_m$ and m is the total number of orders which describes the

maximum embedded depth of a Hilbert curve. An order corresponds to a specific bit of a coordinate dimension or 3 specific consecutive bits of a Hilbert code. Let X_i denotes the i th order of dimension X , and let $\tilde{X}_i = X_1 X_2 \dots X_i$ denotes the partial value of the first i orders of X . For simplicity, we denote (X_i, Y_i, Z_i) as the coordinate of the i th order. Let the Hilbert code of P is $A = a_1 a_2 a_3 \dots a_{3i-2} a_{3i-1} a_{3i} \dots a_{3m-2} a_{3m-1} a_{3m}$, where $A_i = a_{3i-2} a_{3i-1} a_{3i}$ is the i th order Hilbert code of A and $\tilde{A}_i = a_1 a_2 a_3 \dots a_{3i-2} a_{3i-1} a_{3i}$ is the partial Hilbert code for the first i orders of A . For the i th order, there is a state T_i corresponding to it. In this paper, T_1 is always set to be 0.

Based on the discussion above, we give the formal definitions of encoding and decoding in Definition 1 and Definition 2.

Definition 1 (Encoding) Given a point $P(X, Y, Z)$, the process of determining the sequence number A on a Hilbert curve for a coordinate P is called encoding.

Definition 2 (Decoding) Given a Hilbert code A , the process of determining the corresponding coordinate $P(X, Y, Z)$ of A is called decoding.

III. Encoding

1. Encoding state views

State views are prime important to our Hilbert encoding and decoding, which reflect the relationships among coordinates, Hilbert codes and the states. In this paper, 2 efficient state views, PHM and PNM, are designed to facilitate encoding. Both PHM and PNM have $12 \times 8 = 96$ entries. Each entry of PHM is a mapping from a physical coordinate of a certain order to the Hilbert code of that order under a certain state. Each entry of PNM is a mapping from a physical coordinate to the state of the next order under a certain state.

Given the 12 basic states, the PHM and PNM can be easily built as shown in Table 1 and Table 2, respectively.

Table 1. PHM

State	Coordinate							
	000	001	010	011	100	101	110	111
0	0	1	3	2	7	6	4	5
1	4	5	7	6	3	2	0	1
2	6	7	5	4	1	0	2	3
3	2	3	1	0	5	4	6	7
4	0	1	7	6	3	2	4	5
5	4	5	3	2	7	6	0	1
6	2	3	5	4	1	0	6	7
7	6	7	1	0	5	4	2	3
8	0	7	3	4	1	6	2	5
9	2	5	1	6	3	4	0	7
10	4	3	5	2	7	0	6	1
11	4	3	7	0	5	2	6	1

Table 2. PNM

State	Coordinate							
	000	001	010	011	100	101	110	111
0	8	4	3	4	10	5	3	5
1	2	4	11	4	2	5	9	5
2	1	7	8	8	1	6	10	10
3	11	11	0	7	9	9	0	6
4	8	0	11	1	6	0	6	1
5	10	10	9	9	0	7	1	7
6	10	10	9	9	4	2	4	3
7	2	8	3	11	2	5	3	5
8	0	2	9	9	4	7	4	7
9	1	3	8	8	1	3	5	6
10	11	11	0	2	5	6	0	2
11	10	10	1	3	4	7	4	7

To reduce the space overheads and improve the encoding efficiency as much as possible, we design PHM and PNM as 2 4D arrays. Given X_i, Y_i, Z_i and T_i of order i , A_i and T_{i+1} can be obtained by retrieving PHM and PNM as shown in Eq.(1) and Eq.(2), respectively.

$$A_i = PHM[T_i][X_i][Y_i][Z_i] \quad (1)$$

$$T_{i+1} = PNM[T_i][X_i][Y_i][Z_i] \quad (2)$$

2. Encoding algorithm

One intuitive way for encoding is to iteratively retrieve the created encoding state views to obtain the Hilbert code and the next state for each order, and then get the final encoding value by combining the encoding values of each order. Taking a coordinate $P(11,5,12)$ and order $m=6$ for an example, a diagram of intuitive iteration is shown in Fig.5.

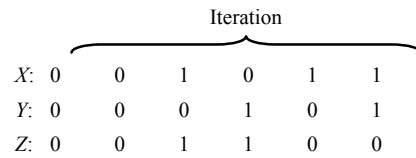


Fig. 5. A diagram of intuitive iteration

The main disadvantage of the above method is that it needs to iteratively access all orders. For the example in Fig.5 and Fig.6, orders need to be accessed which is not efficient. To improve the encoding efficiency, based on thoroughly analyzing the encoding process, Theorem 1 is proposed in this paper as shown below.

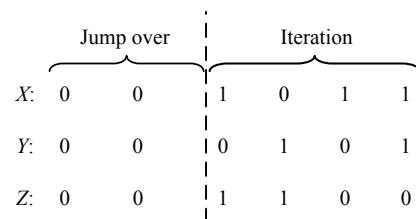


Fig. 6. Skipping the first k orders

Theorem 1 If $\tilde{X}_k$, $\tilde{Y}_k$ and $\tilde{Z}_k$ are all 0s, then $\tilde{A}_k$ is 0, and T_{k+1} is either 0 (k is even) or 8 (k is odd).

The proof of Theorem 1 can be seen in Appendix A. It can be seen from Theorem 1 that if the first k orders of X , Y , and Z are all 0s, $\tilde{A}_k$ and T_{k+1} can be obtained directly, so the first k orders can be skipped safely, thus improving the efficiency. For example, for a coordinate $P(11,5,12)$ and $m = 6$ shown in Fig.6, as $\tilde{X}_2$, $\tilde{Y}_2$ and $\tilde{Z}_2$ are all 0, so we can get $\tilde{A}_2 = 0$ and $T_3 = 0$ directly and skip the first 2 orders. In this way, only 4 orders need to be iterated which is less than the intuitive method. As a result, the encoding efficiency is improved.

How to efficiently detect maximum k for any coordinates is key to efficient encoding. Intuitively, we can check each dimension of a coordinate order-by-order to get k . However, it is not efficient for too many orders need to be processed. To deal with this problem, we introduce an efficient first-1-check (F1C) algorithm^[28] to detect the first 1 in a given number quickly in this paper. F1C introduces a binary-search-like method and executes much faster than the order-by-order manner.

It should be noted that there is no need to execute F1C on all 3 dimensions of a coordinate P . Accordingly, only a single F1C on $\max(X, Y, Z)$ need to be executed, thereby further improving the detection efficiency.

Based on the analysis above, a 3D Hilbert encoding algorithm—JFK-3HE is proposed, which can skip the first k orders efficiently. The complete description JFK-3HE is shown in Fig.7.

```
JFK-3HE ( $X, Y, Z, m$ ) {
// $X, Y, Z$ : coordinate
// $m$ : order
 $A=0$ ; //Hilbert code
 $p=F1C(\max(X, Y, Z))$ ;
 $k=m-p-1$ ;
 $T_{k+1}=k\%2<<3$ ;
for ( $i=k+1$ ;  $i\leq m$ ;  $i++$ ) {
 $A=A<<3 \mid PHM[T_i][X_i][Y_i][Z_i]$ ;
 $T_{i+1}=PNM[T_i][X_i][Y_i][Z_i]$ ;
}
}
```

Fig. 7. JFK-3HE algorithm

Time complexity: The main overhead of JFK-3HE is the iterative lookup operations in the state views. For a coordinate $P(X, Y, Z)$, the lookup operations need to be executed $d = m - k = \log_2(\max(X, Y, Z))$ times, and 2 lookup operations are performed for each iteration, so the time complexity of the algorithm is $O(d)$.

Space complexity: The main space overheads are the spaces for the 2 state views. As mentioned earlier, both PHM and PNM have 96 entries, each entry can be expressed by a byte. So, the total space overhead is 192 B.

From the analysis above, the more 0s in front of an input data are, the smaller d is, so JFK-3HE can work

well on skew distribution scenarios where more data is distributed near the origin.

IV. Decoding

1. Decoding state views

Decoding is the reverse operation of encoding. To achieve efficient decoding algorithm, 2 efficient decoding state views, HPM and HNM, are designed. Both HPM and HNM have $12 \times 2^3 = 96$ entries. Each entry of HPM is a mapping from a Hilbert code of a certain order to the physical coordinate of that order under a certain state. For ease of use in HPM, the coordinate of the i -th order is organized in the form of $X_i Y_i Z_i$, in which each component of it can be easily separated by bit-shift operations. Each entry of HNM is a mapping from a Hilbert code to the state of the next order under a certain state.

Given the 12 basic states, the HPM and HNM can be easily built as shown in Table 3 and Table 4, respectively.

Table 3. HPM

State	Encoding value							
	000	001	010	011	100	101	110	111
0	000	001	011	010	110	111	101	100
1	110	111	101	100	000	001	011	010
2	101	111	110	100	000	010	011	001
3	011	001	000	010	110	100	101	111
4	000	001	101	100	110	111	011	010
5	110	010	011	111	101	001	000	100
6	101	001	000	100	110	010	011	111
7	011	010	110	111	101	100	000	001
8	000	100	110	010	011	111	101	001
9	110	100	000	010	011	001	101	111
10	101	111	011	001	000	010	110	100
11	011	111	101	001	000	100	110	010

Table 4. HNM

State	Encoding value							
	000	001	010	011	100	101	110	111
0	8	4	4	3	3	5	5	10
1	9	5	5	2	2	4	4	11
2	6	10	10	1	1	8	8	7
3	7	11	11	0	0	9	9	6
4	8	0	0	6	6	1	1	11
5	1	9	9	7	7	10	10	0
6	2	10	10	4	4	9	9	3
7	11	3	3	5	5	2	2	8
8	0	4	4	9	9	7	7	2
9	5	1	1	8	8	3	6	6
10	6	2	2	11	11	0	0	5
11	3	7	7	10	10	4	4	1

To reduce the space overheads and improve the decoding efficiency, this paper designs HPM and HNM as 2 2D arrays. For HPM, given the Hilbert code A_i and the state T_i of order i , $X_i Y_i Z_i$ and T_{i+1} can be obtained by retrieving HPM and HNM according to Eq.(3)

and Eq.(4), respectively.

$$X_i Y_i Z_i = HPM[T_i][A_i] \quad (3)$$

$$T_{i+1} = HNM[T_i][A_i] \quad (4)$$

2. Decoding algorithm

As decoding in an order manner is not efficient, to improve the decoding efficiency, Theorem 2 is proposed as shown below.

Theorem 2 If $\tilde{A}_k$ is 0, then $\tilde{X}_k, \tilde{Y}_k, \tilde{Z}_k$ are all 0 and T_{k+1} is either 0 (k is even) or 8 (k is odd).

The proof of Theorem 2 can be seen in Appendix A. It can be seen from Theorem 2 that if $\tilde{A}_k$ is 0, then $\tilde{X}_k, \tilde{Y}_k, \tilde{Z}_k$ and T_{k+1} can be obtained directly, so the first k orders can be skipped safely, thus improving the efficiency. Fig.8 shows an example for a Hilbert code $A = 3224$ and $m = 6$. As $\tilde{A}_2$ is 0, so $\tilde{X}_2, \tilde{Y}_2, \tilde{Z}_2$ are all 0 and T_3 is 0. As a result, the first 2 orders can be skipped and only 4 orders need to be processed. Thus the decoding efficiency is improved.

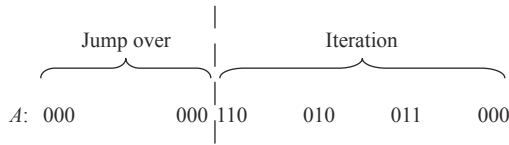


Fig. 8. Skipping the first k orders iteration

To efficiently detect the maximum value of k for a Hilbert code, in this paper, we also use F1C to detect the first 1 in A .

Based on the analysis above, we propose a 3D Hilbert decoding algorithm-JFK-3HD, which can skip the first k orders efficiently. The complete description of JFK-3HD is shown in Fig.9.

```
JFK-3HD ( $A, m$ ) {
//A: Hilbert code
//m: order
 $X=0, Y=0, Z=0;$ 
 $P=F1C(A);$ 
position is 0
 $k=m-p/3-1;$ 
 $T_{k+1}=k\%2<<3;$ 
for ( $i=k+1; i\leq m; i++$ ) {
 $X_i Y_i Z_i = HPM[T_i][A_i];$ 
 $X=X<<1|X_i;$ 
 $Y=Y<<1|Y_i;$ 
 $T_{i+1}=HNM[T_i][A_i]$ 
}
```

Fig. 9. JFK-3HD algorithm

Time complexity: Just like JFK-3HE, retrieving in state views is also the main overhead in JFK-3HD. For a Hilbert code A , it needs to iterate $d = m - k = \log_2(A)/3+1$ times, and 2 lookup operations are performed for each iteration, so the time complexity of JFK-3HD is $O(d)$.

Space complexity: The main space overheads are the spaces for the 2 state views. As mentioned earlier,

both HPM and HNM have 96 entries, each entry can be expressed by a byte. So, the total space overhead is 192 B.

V. Experiments

1. Basic settings

All our experiments were conducted on a PC with Intel(R) Core(TM) i5-6500 CPU @ 3.20 GHz and 8 GB of memory running Win10 64-bit. The compiler we used is Microsoft Visual Studio C++ 2015.

We generate data for both uniform and skewed distribution to examine the effects of data distributions. For uniform distribution, we randomly generate 10^6 coordinates (for encoding) or Hilbert codes (for decoding) for order $m = 4, 8, 16, 32$, respectively. For skewed distribution, we also generate 10^6 coordinates (for encoding) or Hilbert codes (for decoding) for $m = 4, 8, 16, 32$, respectively. To evaluate the skewness of the generated data, we introduce 2 additional parameters: orders of the skewness (θ) and ratio of the skewness (ϕ), where θ is the number of first all 0 orders to the left side of an input data and ϕ is the percentage of the data whose first θ orders are all 0s over the total number of data. For example, $\theta = 8$ and $\phi = 40\%$ represent 40% of the total data with the first 8 orders 0.

To investigate the effectiveness of our algorithms, we compare them with 3 classic 3D encoding and decoding algorithms, e.g., Li^[26], Zhang^[27], and Moore^[25]. As we mainly care about 3D space with equal length dimensions, so for Zhang, we only compare with its Q_{EEE} case.

2. Encoding

1) Uniform distribution

To investigate the encoding efficiency of different algorithms under uniform distribution, JFK-3HE was compared with the 3 encoding algorithms mentioned above for $m = 4, 8, 16, 32$. The results are shown in Fig.10.

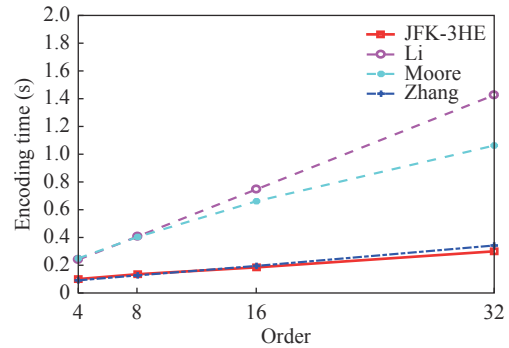


Fig. 10. The comparisons of encoding time under uniform distribution

It can be seen from Fig.10 that the encoding times of all algorithms increase with the increase of orders. The efficiency of JFK-3HE is higher than all other algorithms except for m less than 16. JFK-3HE is 2–4

times faster than Li's and Moore's indicates that state views improve the encoding efficiency significantly.

2) Skewed distribution

To investigate the effects of θ and ϕ , 2 experiments are conducted in this paper: 1) Fix $\theta = 8$, and vary ϕ from 30% to 100%; 2) Fix $\phi = 50\%$, and vary θ from 8 to 20. The corresponding results are shown in Fig.11 and Fig.12, respectively.

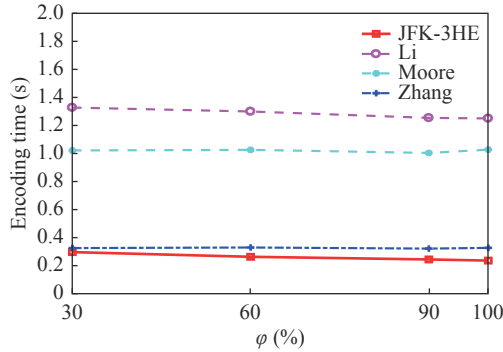


Fig. 11. The effects of ϕ on encoding efficiency

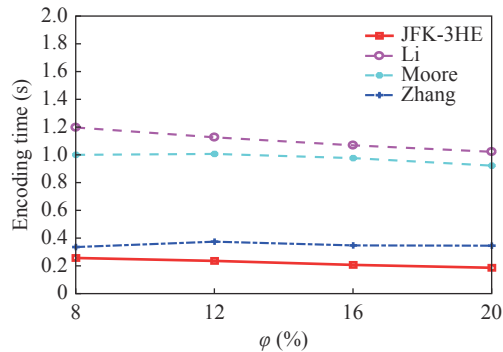


Fig. 12. The effects of θ on coding efficiency

It can be seen from Fig.11 that the efficiency of JFK-3HE is higher than the other 3 algorithms for all ϕ s. More importantly, the encoding time of JFK-3HE decreases with the increase of ϕ , while there are no obvious decreasing trend for the other 3 algorithms. This is because more data can benefit from skipping the first 0s as ϕ increase for JFK-3HE, while the other algorithms cannot benefit from this. The encoding time varies from 0.30 s at $\phi = 30\%$ to 0.24 s at $\phi = 100\%$, a 20% efficiency gain can be gotten. When $\phi = 100\%$, JFK-3HE only needs 0.24 s to encode 1 million 3D coordinates, which is 27% faster than the best one of the other 3 algorithms (Zhang's).

From Fig.12, we can see JFK-3HE is the most efficient among all the encoding algorithms. The bigger θ is, the lower the encoding time will be. When $\theta = 20$, JFK-3HE only needs 0.19 s to encode 1 million 3D coordinates, which is 35% higher than Zhang's, the best of the other 3 algorithms.

3. Decoding

1) Uniform distribution

To investigate the decoding efficiency of different

algorithms under uniform distribution, JFK-3HD was compared with the 3 decoding algorithms for $m = 4, 8, 16, 32$, respectively. The results are shown in Fig.13.

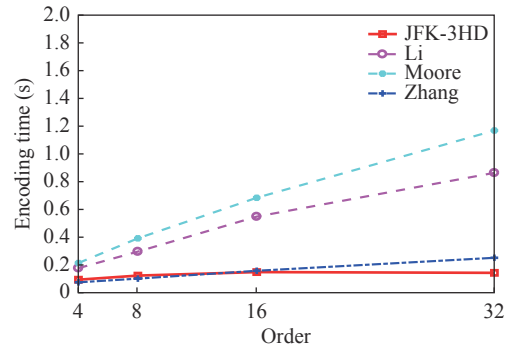


Fig. 13. The comparisons of decoding time under uniform distribution

From Fig.13, a similar trend can be observed as in encoding. The efficiency of JFK-3HD is higher than all other algorithms except for m less than 16. The efficiency of JFK-3HD is much higher than Li's and Moor's indicates that state views can improve decoding significantly.

2) Skew distribution

As in encoding, to investigate the effects of θ and ϕ , 2 experiments are set up in this paper: 1) Fix $\theta = 8$, and vary ϕ from 30% to 100%, 2) Fix $\phi = 50\%$, and vary θ from 8 to 20. The corresponding results are shown in Fig.14 and Fig.15, respectively.

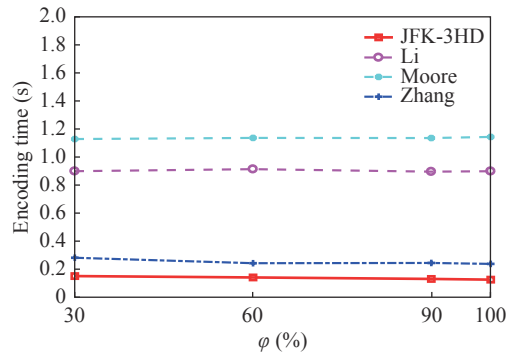


Fig. 14. The effects of ϕ on decoding efficiency

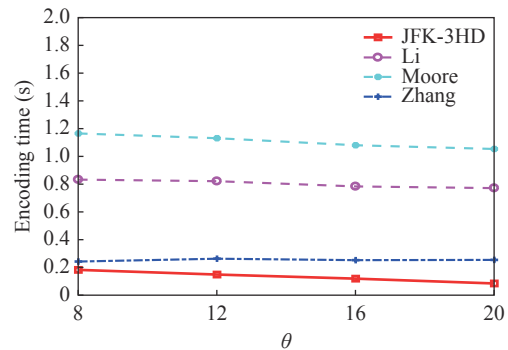


Fig. 15. Influence of θ on decoding efficiency

It can be seen from Fig.14 that the decoding time of JFK-3HD decreases with the increase of ϕ and JFK-

3HD is the highest among all decoding algorithms. This is because more data can benefit from skipping the first 0s as ϕ increase for JFK-3HD, while the other algorithms can't benefit from this. When $\phi = 100\%$, JFK-3HD only needs 0.12 s to decode 1 million Hilbert codes, which is 50% faster than the best of the other 3 algorithms (Zhang's).

From Fig.15, we can see the decoding time of JFK-3HD decreases with the increase of θ and JFK-3HD is the highest among all decoding algorithms. We can benefit from skipping more 0s as θ increase, thus improving the efficiency. When $\theta = 20$, JFK-3HD only needs 0.09 s to decode 1 million Hilbert code, which is 65% higher than the best of the other 3 algorithms (Zhang's).

VI. Conclusions

To improve encoding and decoding efficiency of 3D Hilbert curve, this paper designs efficient encoding and decoding state views. Based on the state views, an efficient encoding algorithm, JFK-3HE, and an efficient decoding algorithm, JFK-3HD, are proposed. JFK-3HE and JFK-3HD can skip the first 0s in input data and avoid iterations order-by-order, thus decreasing the encoding and decoding complexity. Experimental results show that these 2 algorithms are especially efficient for skewed distribution and outperform the state-of-the-art algorithms significantly.

One weakness of these 2 algorithms lies in that they are mainly efficient in skewed scenarios where the first k orders are all 0s. In the future, we will try to investigate ways to make it suitable for other skewed scenarios. Extending them to irregular 3D or high-dimensional space will also be our future research directions.

Appendix A

Proof of Theorem 1 When $k = 1$, as X_1, Y_1, Z_1 and T_1 are all 0. So, according to Table 1 and Table 2, we have $A_1 = PHM[T_1][X_1][Y_1][Z_1] = 0$, $T_2 = PNM[T_1][X_1][Y_1][Z_1] = 8$, so Theorem 1 holds in this case.

When $k = 2$, as the case for the 1st order has been proved, we only consider the 2nd order. As $T_2 = 8$, X_2, Y_2 and Z_2 are all 0, according to Table 1 and Table 2, we have $A_2 = PHM[T_2][X_2][Y_2][Z_2] = 0$, $T_3 = PNM[T_2][X_2][Y_2][Z_2] = 0$, so Theorem 1 holds in this case.

When $k = 3$, similarly, we only consider the 3rd order. As $X_3 = 0, Y_3 = 0, Z_3 = 0$ and $T_3 = 0$, so $A_3 = PHM[T_3][X_3][Y_3][Z_3] = 0$ and $T_4 = PNM[T_3][X_3][Y_3][Z_3] = 8$, which means the input and output of the 3rd order are exactly the same as that of the 1st order; This makes the input and output the 4th order are the same as the 2nd order for $k = 4$. As a result, the execution process loops as shown in Fig.A-1, so Theorem 1 holds in the case of $k \geq 3$.

Therefore, Theorem 1 holds.

```

X:  0  0  0  0  ...
Y:  0  0  0  0  ...
Z:  0  0  0  0  ...
State: 0---8---0---8-- ...
A:   000-000-000-000- ...

```

Fig. A-1. Diagram of the looping encoding process

Proof of Theorem 2 When $k = 1$, as $T_1 = 0$ and $A_1 = a_1 a_2 a_3 = 000$, we have $X_1 Y_1 Z_1 = HPM[T_1][A_1] = 0$ and $T_2 = HNM[T_1][A_1] = 8$ by retrieving Table 3 and Table 4, respectively. So Theorem 2 holds in this case.

When $k = 2$, as the case for the 1st order has been proved, we only consider the 2nd order. As $T_2 = 8$ and $A_2 = a_4 a_5 a_6 = 000$, we have $X_2 Y_2 Z_2 = HPM[T_2][A_2] = 0$ and $T_3 = HNM[T_2][A_2] = 0$ by retrieving Table 3 and Table 4, respectively. So Theorem 2 holds in this case.

When $k = 3$, similarly, we only consider the 3rd order. As $A_3 = a_7 a_8 a_9 = 000$, we have $X_3 Y_3 Z_3 = HPM[T_3][A_3] = 0$ and $T_4 = HNM[T_3][A_3] = 8$, which means the input and output of the 3rd order are exactly the same as that of the 1st order; And this makes the input and output the 4th order are the same as the 2nd order for $k = 4$. As a result, the execution process loops as shown in Fig.A-2, so Theorem 2 holds in the case of $k \geq 3$.

Therefore, Theorem 2 holds.

```

State: 0---8---0---8-- ...
A:   000-000-000-000- ...
X:  0  0  0  0  ...
Y:  0  0  0  0  ...
Z:  0  0  0  0  ...

```

Fig. A-2. Diagram of the looping decoding process

References

- [1] G. Peano, "Sur une courbe, qui remplit toute une aire plane," *Mathematische Annalen*, vol.36, pp.157–160, 1890.
- [2] N. Chen, N. Wang and B. Shi, "A new algorithm for encoding and decoding the Hilbert order," *Software Practice & Experience*, vol.37, no.8, pp.897–908, 2007.
- [3] S. Ghosh and S. Bhattacharya, "Hilbert curve based steganographic scheme for large data hiding," *Third Int. Conf. on Image Information Processing*, pp.91–96, 2015.
- [4] L. F. Cardona and L. E. Múnera, "Self-similarity of space filling curves," *Ingeniería y competitividad*, vol.18, pp.113–124, 2016.
- [5] L. Chen, Y. Gao, X. Li, et al., "Efficient metric indexing for similarity search and similarity joins," *IEEE Transactions on Knowledge and Data Engineering*, vol.29, no.3, pp.556–571, 2017.
- [6] P. Nagarkar, K. S. Candan and A. Bhat, "Compressed spatial hierarchical bitmap (cSHB) indexes for efficiently processing spatial range query workloads," *Proceedings of the Vldb Endowment*, vol.8, pp.1382–1393, 2015.
- [7] H. Singh and S. Bawa, "A survey of traditional and MapReduceBased spatial query processing approaches," *ACM SIGMOD Record*, vol.46, pp.18–29, 2017.
- [8] H. N. Rad and F. Karimipour, "Representation and generation of space-filling curves: A higher-order functional approach," *Journal of Spatial Science*, vol.66, no.3, pp.459–

- 479, 2019.
- [9] G. Schrack and L. Stocco, "Generation of spatial orders and space-filling curves," *IEEE Transactions on Image Processing*, vol.24, no.6, pp.1791–1800, 2015.
 - [10] P. Murali and V. Sankaradass, "An efficient space filling curve based image encryption," *Multimedia Tools and Applications*, vol.78, no.2, pp.2135–2156, 2019.
 - [11] J. Weissenböck, B. Fröhler, E. Gröller, *et al.*, "Dynamic volume lines: Visual comparison of 3D volumes through space-filling curves," *IEEE Transactions on Visualization and Computer Graphics*, vol.25, no.1, pp.1040–1049, 2019.
 - [12] H. Liu, K. Wang, B. Yang, *et al.*, "Load balancing using hilbert space-filling curves for parallel reservoir simulations," *arXiv preprint*, arXiv:1708.01365, 2017.
 - [13] Abduljabbar M., Markomanolis G.S., Ibeid H., *et al.*, "Communication reducing algorithms for distributed hierarchical N-body problems with boundary distributions," *ISC High Performance 2017: High Performance Computing*, vol.10266, pp.79–96, 2017.
 - [14] E. R. Tyercha, G. S. Kazmaier, H. Gildhoff, *et al.*, "Querying spatial data in column stores using tree-order scans," *Patent*, Germany, US09613055B2, 2017-04-04.
 - [15] Y. W. Dai, G. J. Liu, and S. G. Ye, "Adaptive steganography based on hilbert filling curve," *Acta Electronica Sinica*, vol.36, no.S1, pp.35–38, 2008. (in Chinese)
 - [16] A. R. Butz, "Alternative algorithm for hilbert's space-filling curve," *IEEE Trans. Comput.*, vol.20, pp.424–426, 1971.
 - [17] X. Cao, G. Wan, and Z. Zhang, "Compact Hilbert curve index algorithm based on gray code," *Acta Geodaetica Et Cartographica Sinica*, vol.45, no.S1, pp.90–98, 2016.
 - [18] S. Li, E. Zhong, S. Wang, *et al.*, "An algorithm for Hilbert ordering code based on state-transition matrix," *Journal of Geo-information Science*, vol.16, no.6, pp.846–851, 2014.
 - [19] J. Burkardt, "Hilbert curve convert between 1D and 2D coordinates of Hilbert curve," available at: http://people.math.sc.edu/Burkardt/c_src/hilbert_curve/hilbert_curve.html, 2015.
 - [20] M. Hu, X. Tian, S. Xia, *et al.*, "Image scrambling based on 3-D Hilbert curve," *2010 3rd International Congress on Image and Signal Processing*, pp.147–149, 2010.
 - [21] I. Al-Kharusi and D. W. Walker, "Locality properties of 3D data orderings with application to parallel molecular dynamics simulations," *Int. Journal of High Performance Computing Applications*, vol.33, no.5, pp.998–1018, 2019.
 - [22] S. Hans, "A three-dimensional Hilbert curve," *International Journal of Mathematical Education in Science and Technology*, vol.24, no.4, pp.541–545, 2010.
 - [23] R. J. Millar, M. E. C. Hull, and J. H. Frazer, "The millar polyhedron and its use in the construction of octrees," *The Computer Journal*, vol.36, no.2, pp.186–194, 1993.
 - [24] A. J. Fisher, "A new algorithm for generating hilbert curves," *Software Practice & Experience*, vol.16, no.1, pp.5–12, 1986.
 - [25] Github, "Fast Hilbert curve generation, sorting, and range queries," available at: <https://github.com/Cheedoong/hilbert>, 2016.
 - [26] C. Li, X. Duan, and Y. Feng, "N-dimensional Hilbert curve generation algorithm," *Chinese Journal of Image Graphics*, vol.11, no.8, pp.1068–1075, 2006.
 - [27] J. Zhang and S. I. Kamata, "A generalized 3-D Hilbert scan using look-up tables," *Journal of Visual Communication and Image Representation*, vol.23, no.3, pp.418–425, 2012.
 - [28] Rosetta, "Find first and last set bit of a long integer," available at: https://rosettacode.org/wiki/Find_first_and_last_set_bit_of_a_long_integer, 2019.



JIA Lianyin was born in 1978. He received his Ph.D. degree from South China University of Technology in 2012. He is currently an Associate Professor in Kunming University of Science and Technology. His research interests include database, data mining, information retrieval, and parallel computing. (Email: lianyinjia@kust.edu.cn)



LIANG Binbin was born in 1993. He received his M.S. degree from Kunming University of Science and Technology in 2021. He is currently working as a Network Engineer in Ruijie Network Co., Ltd. His research interests include database and data mining. (Email: 2041429702@qq.com)



(Email: lmjlykm@163.com)

LI Mengjuan was born in 1983. She received her M.S. degree in computer science from Kunming University of Science and Technology in 2008. She is currently a Librarian in the Department of Technology, Library, Yunnan Normal University, Kunming, China. Her main research interests include information retrieval and parallel computing.



LIU Yong was born in 1973. He received his Ph.D. degree in engineering from South China University of Technology in 2013. His research interests include bioinformatics, deep learning, and high-performance computing. (Email: niu20060040@gmail.com)



He received B.S. and M.S. degrees from Chongqing University in computer science in 1982 and 1984, respectively. He received Ph.D. degree from Karlsruhe Institute of Technology, Germany, in 1993. He is currently a Principal Lecturer in the School of Computing and Augmented Intelligence at Arizona State University, USA. His research interests are in service-oriented computing, big data processing, and artificial intelligence. (Email: yinong@asu.edu)



DING Jiaman (corresponding author) was born in 1974. He received M.S. degree from Kunming University of Science and Technology in 2005. He is currently an Associate Professor in Kunming University of Science and Technology. His research interests include data mining and cloud computing. (Email: jiamanding@kust.edu.cn)